

PLAN Y REPORTE DE PRUEBAS DE SOFTWARE — SPRINT 4

Foco del Incremento: Flujo completo de reservas, historial personal, botón de WhatsApp y notificación por email **Enfoque de Testing:** Tests unitarios (JUnit 5 + Mockito / Vitest + React Testing Library), Tests de integración (MockMvc + Testcontainers), Verificación de UI Manual, Suite E2E con Playwright (agregado complementario)

1. Matriz Detallada de Casos de Prueba (Test Cases)

TC-30: Seleccionar Fecha y Autenticación (US #30)

- **Historias de Usuario Asociadas:** US #30 (Seleccionar fecha de check-in/check-out)
- **Precondiciones:** Servidor backend activo. Al menos un alojamiento con reservas CONFIRMED existentes en BD.
- **Tipos de Verificación:** UI Manual, Test de Integración Automatizado (JUnit 5 + MockMvc + Testcontainers).

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	Click "Reservar" en ProductDetail sin login	Aparece link a /login en lugar del botón de reserva	✓ Pasa
2	Acceder a /booking/:id sin sesión activa	RequireAuth redirige a /login con mensaje de autenticación	✓ Pasa
3	Mensaje en /login incluye mención a registro	Texto "Necesitás iniciar sesión para continuar. Si no tenés cuenta, podés registrarte." visible	✓ Pasa
4	Login exitoso desde el flujo de redirección	Vuelve automáticamente a /booking/:id (state from preservado)	✓ Pasa
5	DatePicker muestra fechas ocupadas deshabilitadas al cargar	GET /api/lodgings/{id}/availability invocado al montar. Rangos CONFIRMED bloqueados visualmente	✓ Pasa
6	Intento de seleccionar fecha dentro de rango ocupado	Fecha no seleccionable. Cursor indica estado deshabilitado	✓ Pasa

TC-31: Visualizar Detalles de Reserva (US #31)

- **Historias de Usuario Asociadas:** US #31 (Visualizar detalles del alojamiento al reservar)
- **Precondiciones:** Usuario autenticado con JWT válido. Alojamiento existente con imagen, descripción y features.
- **Tipos de Verificación:** UI Manual.

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	Navegar a <code>/booking/:id</code> como usuario autenticado	Columna izquierda muestra nombre y ciudad del alojamiento	✓ Pasa
2	Verificar precio por noche en columna izquierda	Precio visible con formato correcto	✓ Pasa
3	Verificar imagen del alojamiento	Imagen cargada. Placeholder visible si falla la carga	✓ Pasa
4	Verificar descripción del alojamiento	Texto de descripción visible	✓ Pasa
5	Verificar lista de features con íconos Lucide	Features visibles con íconos SVG	✓ Pasa
6	Datos del usuario pre-cargados (nombre, apellido, email)	Campos read-only con los datos del usuario autenticado	✓ Pasa
7	Campo teléfono en formulario	Input activo y editable	✓ Pasa
8	DatePickers con fechas pre-cargadas desde <code>ProductDetail</code>	Fechas seleccionadas previamente visibles en los pickers	✓ Pasa
9	Cambiar fechas en el formulario	Total estimado se recalcula dinámicamente ($\text{días} \times \text{pricePerNight}$)	✓ Pasa

TC-32: Realizar Reserva (US #32)

- **Historias de Usuario Asociadas:** US #32 (Realizar reserva y recibir confirmación)
- **Precondiciones:** Usuario autenticado con JWT válido. Alojamiento existente sin solapamiento de fechas en el rango elegido.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend), Test Unitario de Componente (frontend), UI Manual.

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	POST /api/reservations con datos válidos y token JWT	HTTP 201 Created. Body con id, fechas, totalPrice, status: CONFIRMED	✓ Pasa
2	POST /api/reservations con fechas solapadas a una reserva CONFIRMED	HTTP 409 Conflict	✓ Pasa
3	POST /api/reservations sin token JWT	HTTP 401 Unauthorized o 403 Forbidden	✓ Pasa
4	Confirmar reserva exitosa desde BookingPage	Navegación a /booking/confirmation con datos de la reserva	✓ Pasa
5	/booking/confirmation muestra nombre del alojamiento, fechas, huésped y total	Todos los datos correctos visibles	✓ Pasa
6	/booking/confirmation muestra mensaje de email enviado	Texto "Te enviamos un email de confirmación" visible	✓ Pasa
7	Link "Ver mis reservas" en la pantalla de confirmación	Navega a /my-reservations	✓ Pasa
8	Acceder a /booking/confirmation directamente sin reserva previa	Redirige a home (/)	✓ Pasa
9	Error al reservar (ej: 409 Conflict)	Mensaje de error específico visible en el formulario	✓ Pasa

Cobertura automatizada (frontend): `BookingConfirmation.test.jsx` — 4 tests unitarios: detalles de reserva (nombre, fechas formateadas `dd/mm/aaaa`, huésped, total), nota de email, links a `/my-reservations` y `/`, redirección a `/` sin state.

TC-33: Acceder al Historial de Reservas (US #33)

- **Historias de Usuario Asociadas:** US #33 (Acceder al historial personal de reservas)
- **Precondiciones:** Usuario autenticado con al menos una reserva creada. Servidor backend activo.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend), Test Unitario de Componente (frontend), UI Manual.

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	GET /api/reservations/my con token JWT válido	HTTP 200. Array con las reservas del usuario autenticado	✓ Pasa
2	GET /api/reservations/my sin token JWT	HTTP 401 Unauthorized o 403 Forbidden	✓ Pasa
3	Acceder a /my-reservations sin sesión activa	RequireAuth redirige a /login	✓ Pasa
4	Lista muestra nombre del alojamiento, ciudad, fechas y estado	Todos los campos visibles y correctos	✓ Pasa
5	Lista muestra datos de contacto y total de la reserva	Teléfono, email y precio total visibles	✓ Pasa
6	Lista ordenada por checkIn descendente	La reserva más reciente aparece primera	✓ Pasa
7	Link "Mis reservas" en el Header para usuario autenticado	Visible para usuarios con sesión activa. Navega a /my-reservations	✓ Pasa
8	Link "Mis reservas" NO visible para usuarios anónimos	No renderiza en header cuando no hay sesión	✓ Pasa
9	GET /api/reservations/my como usuario A — no retorna reservas de usuario B	Solo se devuelven las reservas del usuario autenticado (aislamiento de datos)	✓ Pasa

Cobertura automatizada (backend): `ReservationServiceImplTest` — `getMyReservations_returnsReservationsMappedToResponse`, `getMyReservations_returnsEmptyListWhenNoReservationsExist`. `ReservationControllerIntegrationTest` — `shouldReturnUserReservationsOrderedByCheckInDesc`, `shouldReturnOnlyAuthenticatedUserOwnReservations`.

Cobertura automatizada (frontend): `MyReservationsPage.test.jsx` — lista con noches calculadas, singular/plural, estado vacío con CTA, error de fetch. `Header.test.jsx` — "Mis reservas" solo para autenticados; login/register para anónimos; logout visible/oculto según sesión.

TC-34: Botón de WhatsApp (US #34)

- **Historias de Usuario Asociadas:** US #34 (Iniciar conversación de WhatsApp con mensaje pre-cargado)
- **Precondiciones:** Variable de entorno `VITE_WHATSAPP_NUMBER` definida en `.env`. Servidor frontend activo.
- **Tipos de Verificación:** UI Manual.

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	Cargar cualquier página pública con <code>VITE_WHATSAPP_NUMBER</code> definida	Botón flotante de WhatsApp visible en esquina inferior derecha	✓ Pasa
2	<code>VITE_WHATSAPP_NUMBER</code> no definida en entorno	Componente <code>WhatsAppButton</code> no renderiza en el DOM	✓ Pasa
3	Click en botón de WhatsApp	Abre <code>wa.me/{número}</code> con mensaje pre-cargado en nueva pestaña	✓ Pasa
4	Verificar acceso sin login	Botón visible para usuarios anónimos. No requiere autenticación	✓ Pasa
5	Hacer scroll vertical en la página	Botón permanece en posición fija (bottom: 24px; right: 24px)	✓ Pasa

Cobertura automatizada (frontend): `WhatsAppButton.test.jsx` — no renderiza con env vacía; URL `wa.me` correcta con número configurado; atributos `target="_blank"` y `rel="norereferrer"`; posicionamiento fijo; sin requisito de autenticación.

TC-35: Email de Confirmación (US #35)

- **Historias de Usuario Asociadas:** US #35 (Recibir email de confirmación de reserva)
- **Precondiciones:** Servidor backend activo. Para SMTP: `MAIL_SMTP_ENABLED=true` y credenciales Mailtrap configuradas en variables de entorno.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado, Verificación de inbox (Mailtrap).

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	POST <code>/api/reservations</code> CON datos válidos (modo default)	<code>EmailService.sendReservationConfirmation()</code> invocado por el servicio	✓ Pasa
2	Revisar consola del servidor con <code>ConsoleEmailServiceImpl</code> activo	Log con datos de la reserva (nombre alojamiento, fechas, huésped, total) visible en stdout	✓ Pasa
3	POST <code>/api/reservations</code> CON <code>MAIL_SMTP_ENABLED=true</code> y credenciales Mailtrap	Email recibido en inbox de Mailtrap	✓ Pasa
4	Verificar contenido del email recibido en Mailtrap	Contiene: nombre del alojamiento, <code>checkIn</code> , <code>checkOut</code> , nombre del huésped, total, estado <code>CONFIRMED</code>	✓ Pasa
5	Verificar destinatario del email	Email enviado a la dirección <code>guestEmail</code> de la reserva	✓ Pasa

TC-36: Edición de Alojamiento en Panel Admin (Mejora complementaria)

- **Historias de Usuario Asociadas:** Mejora complementaria — edición de alojamientos en admin
- **Precondiciones:** Usuario con rol ADMIN autenticado. Al menos un alojamiento existente en BD.
- **Tipos de Verificación:** API Rest, UI Manual.

Paso	Acción / Estímulo de Prueba	Resultado Esperado (Criterio de Aceptación)	Estado
1	Tabla de alojamientos en panel admin	Botón "Editar" visible en cada fila	✓ Pasa
2	Click en "Editar" de un alojamiento	LodgingFormModal se abre con título "Editar alojamiento" y campos pre-llenados	✓ Pasa
3	Verificar datos pre-llenados	Nombre, ciudad, descripción, precio, capacidad y demás campos con los valores actuales	✓ Pasa
4	Modificar nombre y hacer click en "Guardar"	PUT /api/lodgings/{id} → HTTP 200. Tabla actualizada con nuevo nombre	✓ Pasa
5	PUT /api/lodgings/{id} sin token ADMIN	HTTP 403 Forbidden	✓ Pasa
6	Abrir modal de edición y cerrar sin modificar nada	Modal cierra sin mostrar ConfirmDialog	✓ Pasa
7	Modificar un campo y click en "Cancelar"	ConfirmDialog aparece preguntando si descartar cambios	✓ Pasa
8	Confirmar descarte en ConfirmDialog	Modal cierra. Alojamiento no modificado en BD	✓ Pasa

TC-37: Suite E2E con Playwright (Agregado complementario)

- **Historias de Usuario Asociadas:** Transversal — cobertura end-to-end de los flujos críticos de la aplicación, incorporada por iniciativa del equipo fuera del alcance original del sprint
- **Precondiciones:** Backend activo en :8080 y frontend en :5173 . Credenciales de usuario de prueba configuradas en e2e/.env .
- **Tipos de Verificación:** Test E2E Automatizado (Playwright, Chromium + Firefox), con Page Object Model y fixtures de autenticación.

Spec	Escenarios	Cobertura	Estado
smoke.spec.js	3	Carga de home con formulario de búsqueda, página de login y página de registro	✓ Pasa
auth.spec.js	4	Login válido (nombre en header), login con contraseña incorrecta (error visible), logout con redirección a home, registro exitoso	✓ Pasa
search.spec.js	2	Búsqueda por ciudad navega a /search, página de resultados renderiza encabezado	✓ Pasa
reservations.spec.js	2	Historial de reservas carga para usuario autenticado, usuario anónimo es redirigido por RequireAuth	✓ Pasa
visual.spec.js	6	Regresión visual contra capturas de referencia: home, login, registro, resultados de búsqueda, detalle de alojamiento y mis reservas	✓ Pasa

Cada escenario se ejecuta en Chromium y Firefox: 17 escenarios × 2 navegadores = **34 ejecuciones, todas en verde.**

2. Resumen de Ejecución

Tipo de Prueba	Cantidad	Estado
Tests Automatizados Backend (JUnit 5 + MockMvc + Testcontainers)	265 tests	✓ Todos pasan
Tests Automatizados Frontend (Vitest + React Testing Library)	232 tests	✓ Todos pasan
Tests E2E Playwright — agregado complementario (Chromium + Firefox)	17 escenarios × 2 navegadores (34 ejecuciones)	✓ Todos pasan
Casos de Prueba Funcionales (Plan)	46 escenarios	✓ 46/46 verificados

3. Cobertura por Historia de Usuario

User Story	Cantidad TC	Tipo	Estado
US #30 — Seleccionar fecha	6 TC	Automatizado (backend + frontend) + Manual	✓ Completo
US #31 — Visualizar detalles	9 TC	Automatizado (frontend) + Manual	✓ Completo
US #32 — Realizar reserva	10 TC	Automatizado (backend + frontend) + Manual	✓ Completo
US #33 — Historial de reservas	9 TC	Automatizado (backend + frontend) + Manual	✓ Completo
US #34 — WhatsApp	5 TC	Automatizado (frontend) + Manual	✓ Completo
US #35 — Email de confirmación	5 TC	Automatizado (backend) + Manual	✓ Completo
TC-36 — Edición admin (complementaria)	8 TC	Automatizado + Manual	✓ Completo
TC-37 — Suite E2E Playwright (agregado)	17 TC	Automatizado E2E	✓ Completo

4. Herramientas Utilizadas

Herramienta	Propósito
JUnit 5 + Mockito	Tests unitarios de servicios
MockMvc + Testcontainers	Tests de integración con MariaDB efímera
Playwright	Tests E2E y regresión visual en Chromium y Firefox (agregado complementario)
Postman	Pruebas manuales de API
Mailtrap (SMTP sandbox)	Verificación de emails de confirmación
Navegador (Chrome)	Verificación de UI
Swagger UI	Documentación y exploración de endpoints

5. Defectos Encontrados y Corregidos

ID	Descripción	Severidad	Estado
BUG-01	BookingPage.jsx prefillaba el teléfono desde data[data.length - 1] (la reserva más antigua), cuando la API retorna por checkIn DESC y la reserva más reciente es data[0]. El test lo describía como "latest prior reservation" pero verificaba el último elemento — inconsistencia entre semántica del test y comportamiento real.	Baja (UX)	✓ Corregido — data[0]; test actualizado para reflejar el orden DESC real de la API.